Comparison of Sentinel-2 Processing Approaches

A choice can be a burden

Chris Reudenbach

2025-12-04

Inhaltsverzeichnis

Overview

This page compares three fundamentally different workflows for obtaining and processing Sentinel-2 time series for mesoscale forest applications (Burgwald AOI, 2018–2022). The goal is to show where each method excels, where the limits are, and why there is no single "best" approach.

The three approaches:

1. **gdalcubes** — high-performance spatiotemporal cube abstraction

2. **CDSE + GetImage** — server-side processing via Copernicus Data Space Ecosystem

3. **Manual terra/raster workflow** — full local control, no abstraction

A key point: **STAC endpoints are not unified**.
Different providers (AWS EarthSearch, CDSE, SentinelHub, Element84) use compatible STAC schemas but *different URL structures, asset names, and filtering logic*. A cross-provider "generic STAC script" is therefore not feasible without provider-specific adapters.

Conceptual Summary

A) gdalcubes

`gdalcubes` provides fast, memory-safe spatiotemporal mosaicking and resampling on-the-fly. It converts a list of STAC items into a 4D cube (x, y, t, band) and applies functions lazily during execution. Performance is excellent and most operations (aggregation, masking, indices) scale linearly with threads.

Trade-off:
Cube creation always implies *aggregation* (dt = monthly, weekly, or a custom interval) and *resampling* to a uniform grid. The result is efficient but no longer identical to original Sentinel-2 granules.

B) CDSE + GetImage (server-side JS scripts)

CDSE executes Javascript processing scripts (NDVI, RawBands, kNDVI, SAVI, EVI…) directly on ESA servers. The user receives minimally-preprocessed imagery at original 10–20 m resolution. Cloud masking, index computation, or mosaicking happen before download.

Trade-off:
High flexibility and high fidelity but always dependent on the availability and correctness of server-side scripts and OAuth token authentication. Performance drops for long time series (each request is a full server job).

C) Manual terra/raster workflow

The manual approach reads each asset (B02, B03, B04, B08, B11, etc.) as a local GeoTIFF and computes indices by hand. It offers complete control over cloud masking, compositing logic, smoothing, and QA.

Trade-off:
Highest transparency, highest reproducibility, but slowest. Large multi-year AOIs lead to long runtimes and high disk load.

Comparison Matrix (1 = poor, 3 = good)

```r
knitr::kable(
  data.frame(
    Criterion = c(
      "Performance (multi-year AOI)",
      "Fidelity to original scenes",
      "Transparency of processing steps",
      "Control over low-level details",
      "Simplicity of code",
      "Didactic value",
      "Time-series / cube analytics",
      "One-off high-quality scenes"
    ),
    gdalcubes = c(3,2,2,2,3,1.5,3,2),
    CDSE_GetImage = c(2,3,2.5,2.5,2,2.5,2,3),
    terra_manual = c(1,3,3,3,1,3,1.5,2.5)
  ),
  caption = "Comparison of Sentinel-2 processing pipelines (1=poor, 3=good)."
)
```

Tabelle 1: Comparison of Sentinel-2 processing pipelines (1=poor, 3=good).

Criterion	gdalcubes	CDSE_GetImage	terra_manual
Performance (multi-year AOI)	3.0	2.0	1.0
Fidelity to original scenes	2.0	3.0	3.0
Transparency of processing steps	2.0	2.5	3.0

Criterion	gdalcubes	CDSE_GetImage	terra_manual
Control over low-level details	2.0	2.5	3.0
Simplicity of code	3.0	2.0	1.0
Didactic value	1.5	2.5	3.0
Time-series / cube analytics	3.0	2.0	1.5
One-off high-quality scenes	2.0	3.0	2.5

Interpretation

- **Best for time series:** `gdalcubes` (performance, cube logic, parallel execution). Suited for large-area monitoring, phenology, anomaly detection, and aggregated indicators.
- **Best for high-fidelity scenes and indices: CDSE + GetImage**. Uses ESA's own processing chain, preserves ~native resolution, and provides server-processed cloud masks and indices.
- **Best for teaching, transparency, and supervision of every step: Manual terra/raster**. Ideal for methodological demonstrations, QA, and thesis contexts where exact control matters more than performance.

Why STAC cannot be unified

Even though STAC is a common standard, **each provider exposes different API patterns**:

- CDSE uses `assets$B04_10m$href` and requires OAuth tokens.
- AWS EarthSearch uses COGs under `sentinel-s2-l2a-cogs`, with different property keys.
- SentinelHub implements a STAC layer but requires preprocessing scripts and authentication.
- Element84 uses separate asset naming and often different temporal coverage.

Therefore, *filtering, asset selection, and pre-processing must be provider-specific.* A universal STAC R script is not feasible without explicit adapters for each backend.

When to choose which method?

Choose gdalcubes if:

- you need long time series (multi-year)
- uniform grid and monthly/weekly aggregates are acceptable
- you want high performance and vectorised EO analysis
- memory constraints matter

Choose CDSE + GetImage if:

- you need *near-original* resolution
- you want server-side processing (NDVI, EVI, SAVI, RawBands)
- local disk usage should stay minimal
- you want ESA's operational QA bands

Choose terra/raster if:

- you need absolute control over masking, resampling, compositing
- you are demonstrating methods in a didactic setting
- transparency and reproducibility matter more than speed

Conclusion

The three workflows are complementary. They represent the fundamental trade-off space in satellite data processing:

- **gdalcubes = performance + cube abstraction**
- **CDSE = fidelity + server-side processing**
- **manual terra = transparency + full control**

In practice, all three are useful depending on whether the objective is: (a) efficient multiyear analysis, (b) exact reproduction of ESA-processed scenes, or (c) methodological inspection of each step.

“‘